

Part3

这一章的主线是：

```
并行编程模型
├─ Implicit Model：隐式并行模型
├─ Data-Parallel Model：数据并行模型
├─ Shared-Variable Model：共享变量模型
└─ Message-Passing Model：消息传递模型

并行编程技术
├─ ANSI X3H5
├─ Pthreads
├─ MPI
├─ PVM
└─ HPF

并行程序支持技术
├─ 并行化编译器
├─ 性能分析
└─ 科学数据可视化
```

1. The Implicit Model：隐式并行模型

第 1 页讲的是 **Implicit Model**。

隐式并行模型的基本思想是：

程序员写普通顺序程序，编译器自动把它转换成并行程序。

也就是说，程序员不用显式写线程、进程、通信和同步，而是依赖编译器发现程序中的并行性。

课件举的例子包括：

- KAP from Kuck and Associates
- FORGE from Advanced Parallel Research

它的优点：

- 语义简单；
- 不容易死锁；
- 程序结果通常确定；
- 因为程序本质上还是单线程控制流，所以测试、调试、正确性验证更容易；
- 顺序程序可移植性好。

但缺点非常关键：

自动并行编译器非常难开发，而且自动并行化效率通常较低。

原因是编译器必须自动判断：

```
哪些循环可以并行？
哪些语句有数据依赖？
哪些变量需要同步？
哪些操作会产生冲突？
```

很多情况下，编译器无法确定程序是否安全并行化，就只能保守处理，导致并行效果不好。

所以隐式模型的核心矛盾是：

```
程序员轻松，但编译器很难；
可移植性好，但性能可能差。
```

2. The Data-Parallel Model：数据并行模型

第 2 页讲 **Data-Parallel Model**。

数据并行模型是 SIMD 机器的天然模型，也可以在 SPMD 上实现。

它的核心思想是：

对大量数据元素执行相同或相似操作。

例如数组加法：

```
for (i = 0; i < n; i++)
    c[i] = a[i] + b[i];
```

每个 `i` 的计算互不影响，因此可以同时执行。

数据并行模型强调：

- local computations，本地计算；
- data routing operations，数据路由操作；
- 对数组整体操作，而不是一个个标量操作。

课件提到的例子：

- Fortran 90
- HPF, High Performance Fortran

数据并行模型的特点：

特点	解释
Single thread	从控制流看，程序像顺序程序
Parallel synchronous operation	并行同步执行
Loosely synchronous	每条语句后有同步
Single address space	所有变量位于单一地址空间
Explicit data allocation	用户可以指定数据如何分布
Implicit communication	用户不用显式写通信

数据并行模型的优点是编程较简单，尤其适合科学计算中的数组、矩阵和网格问题。

它的缺点是灵活性不如消息传递模型，适合规则数据并行，不适合复杂不规则通信。

3. The Shared-Variable Model：共享变量模型

第 3 页讲 **Shared-Variable Model**。

共享变量模型是 PVP、SMP、DSM 机器的天然编程模型。

它的基本思想是：

多个线程通过读写共享变量进行通信。

程序员看到的是一个共享地址空间，多个线程可以访问同一个变量。

例如：

```
shared int sum;

Thread 1: sum += local1;
Thread 2: sum += local2;
```

这种模型中，线程之间不需要显式发送消息，而是通过共享内存间接通信。

课件指出，ANSI X3H5 是一个相关标准，但程序的可移植性仍然是问题。

共享变量模型的特点：

特点	解释
Multiple threads	多线程，可以是 SPMD 或 MPMD
Asynchronous	每个线程按自己的速度执行
Explicit synchronization	需要显式同步，如 barrier、lock、critical region、event
Single address space	单一地址空间

特点	解释
Implicit data/computation distribution	数据和计算分布通常由系统处理
Implicit communication	通信通过共享变量读写完成

它的优点：

- 编程直观；
- 适合共享内存机器；
- 不需要显式 send/receive。

它的缺点：

- 容易出现数据竞争；
- 同步错误会导致死锁或不确定结果；
- 在分布式系统上实现共享变量开销较大；
- 可移植性较差。

4. The Message-Passing Model：消息传递模型

第 4 页讲 **Message-Passing Model**。

消息传递模型是 MPP 和 COW/Cluster 的天然模型。

核心思想是：

每个进程有自己的地址空间，进程之间通过发送和接收消息通信。

典型代码形式是：

```
send(data, destination);
recv(data, source);
```

课件提到，PVM 和 MPI 极大增强了消息传递程序的可移植性。

消息传递模型的特点：

特点	解释
Multiple threads/processes	多进程或多线程
SPMD / MPMD	可以同程序多数据，也可以多程序多数据
Asynchronous	不同进程以不同速度执行
Explicit synchronization	使用 barrier、lock、critical region、event 等
Multiple address space	每个进程有独立地址空间
Explicit data mapping	程序员显式分配数据和工作量
Explicit communication	程序员显式写消息传递操作

消息传递模型的优点：

- 非常适合分布式内存系统；
- 可扩展性强；
- 数据位置和通信关系清楚；
- MPI 已成为主流标准。

缺点：

- 编程复杂；
- 需要显式管理通信；
- 容易出现通信匹配错误、死锁、负载均衡等问题。

5. 并行编程模型对比

第 5 页用表格比较了四种模型。

可以整理为：

模型	架构	控制线程	计算方式	同步	通信	地址空间	数据分配	可移植性	编程难度
Implicit	SIMD	单线程	同步	自动	隐式	全局	隐式	较好	最简单
Data-Parallel	SIMD	单线程	同步	较松同步	隐式	单地址空间	显式	好	较简单
Shared-Variable	SMP/DSM	多线程	异步	显式	隐式	单地址空间	隐式	较差	较难
Message-Passing	MPP/COW	多进程	异步	显式	显式	多地址空间	显式	较好	较难

可以这样记：

Implicit：程序员最省事，编译器最难。
Data-Parallel：适合数组和规则数据。
Shared-Variable：适合共享内存机器。
Message-Passing：适合分布式内存机器。

6. π Computation：计算 π 的例子

第 6 页给出一个顺序 C 程序，用数值积分计算 π 。

公式是：

$$\pi = \int_0^1 \frac{4}{(1+x^2)} dx$$

用矩形法近似：

$$\pi \approx \sum \frac{4}{(1+x_i^2)} \times \Delta x$$

其中：

$$\begin{aligned} \Delta x &= 1 / N \\ x_i &= (i + 0.5) / N \end{aligned}$$

顺序代码大意是：

```
#define N 1000000

double local, pi = 0.0, w;
w = 1.0 / N;

for (i = 0; i < N; i++) {
    local = (i + 0.5) * w;
    pi = pi + 4.0 / (1.0 + local * local);
}

printf("pi is %f\n", pi * w);
```

这个例子后面会分别用共享变量、MPI、PVM 等方式并行化。

它是并行编程教学中很经典的例子，因为循环迭代之间基本独立，非常适合并行求和。

7. ANSI X3H5：共享内存编程标准

第 7 页讲 ANSI X3H5。

它主要用于共享内存并行编程。

核心概念是 **parallel construct**，即并行构造。

在 parallel construct 内部可以包含：

- parallel block，并行块；
- parallel loop，并行循环；
- single process，单进程段。

课件图中展示了一种结构：

```
program main
  A
  parallel
    B
    psections
      section C
      section D
    end psections
    psingle E
    end psingle
    pdo i = 1, 6
      F(i)
    end pdo no wait
    G
  end parallel
  H
end
```

可以理解为：

- `parallel` 开始后进入并行区域；
- `psections` 中的 C、D 可以由不同线程执行；
- `psingle` 表示 E 只由一个线程执行；
- `pdo` 表示循环 F(i) 被多个线程分担；
- `end parallel` 后回到顺序执行。

ANSI X3H5 中还有隐式 barrier，也叫 fence operation。

在以下位置会强制内存一致：

```
parallel
end parallel
end psection
end pdo
end psingle
```

同步变量有四类：

- Latch
- Lock
- Event
- Ordinal

这部分和后来的 OpenMP 思想很像：
用编译指令或结构化语法表达并行区域、并行循环和同步。

8. POSIX Threads: Pthreads

第 8 页讲 Pthreads。

Pthreads 是 IEEE 标准委员会建立的线程标准，类似 Solaris Threads。

它是共享内存多线程编程的重要接口。

8.1 线程管理函数

课件列出常见函数：

函数	作用
<code>pthread_create</code>	创建线程
<code>pthread_exit</code>	线程退出
<code>pthread_join</code>	等待某个线程结束
<code>pthread_self</code>	返回当前线程 ID

例如：

```
pthread_create(&thread_id, &attr, function, arg);
```

表示创建一个线程，让它执行 `function(arg)`。

8.2 同步函数

课件列出 mutex 和 condition variable。

Mutex，互斥锁

函数	作用
<code>pthread_mutex_init</code>	创建互斥变量
<code>pthread_mutex_destroy</code>	销毁互斥变量
<code>pthread_mutex_lock</code>	加锁
<code>pthread_mutex_trylock</code>	尝试加锁
<code>pthread_mutex_unlock</code>	解锁

mutex 用于保护临界区，例如多个线程共同更新 `sum` 时，需要加锁。

Condition Variable，条件变量

函数	作用
<code>pthread_cond_init</code>	创建条件变量
<code>pthread_cond_destroy</code>	销毁条件变量
<code>pthread_cond_wait</code>	等待条件
<code>pthread_cond_timedwait</code>	限时等待
<code>pthread_cond_signal</code>	唤醒一个等待线程
<code>pthread_cond_broadcast</code>	唤醒所有等待线程

条件变量用于线程之间等待某个事件发生。

9. 共享变量方式计算 π

第 9 页给出 C-like 共享变量并行代码。

主要思想是：

多个线程并行计算不同 `i` 的积分项；
每个线程得到 `local`；
最后通过 `critical` 区域累加到共享变量 `pi`。

代码结构大致是：

```
#pragma parallel
#pragma shared(pi, w)
#pragma local(i, local)
{
    #pragma pdo
    for (i = 0; i < N; i++) {
        local = (i + 0.5) * w;
        local = 4.0 / (1.0 + local * local);
    }

    #pragma critical
    pi = pi + local;
}
```

这里要注意几个点：

- `pi` 和 `w` 是共享变量；
- `i` 和 `local` 是每个线程私有变量；

- `pdo` 把循环分给多个线程;
- `critical` 保证多个线程不会同时更新 `pi`。

为什么 `pi = pi + local` 需要 `critical`?

因为如果两个线程同时执行:

```
pi = pi + local;
```

可能发生数据竞争, 导致某些加法结果丢失。

所以共享变量模型中, 同步非常重要。

10. MPI: Message Passing Interface

第 10 页开始讲 MPI。

MPI 是消息传递编程接口, 是并行计算中最重要的标准之一。

MPI 的基本思想:

多个进程运行顺序语言写成的程序, 并通过 MPI 函数库进行消息发送和接收。

也就是说, MPI 程序通常仍然用 C、C++ 或 Fortran 写, 但加入 MPI 函数调用。

10.1 MPI 程序的计算模型

课件说, MPI 计算由一个或多个 heavyweight processes 组成。

这些进程:

- 通常数量固定;
- 有各自独立地址空间;
- 通过 MPI 库函数通信。

10.2 MPI 通信机制

MPI 通信分为两大类:

类型	说明
Point-to-point	点对点通信, 如 send/receive
Collective communication	集体通信, 如 broadcast、summation、reduce

例如:

```
MPI_Send(...)  
MPI_Recv(...)  
MPI_Bcast(...)  
MPI_Reduce(...)
```

10.3 Communicator

MPI 中有 communicator, 通信子。

它用于定义一组进程, 以及这些进程之间的通信上下文。

最常见的是:

```
MPI_COMM_WORLD
```

表示所有 MPI 进程组成的默认通信域。

课件强调, 虽然 MPI 有 200 多个函数, 但只用六个基本函数就可以解决很多问题。

11. MPI Basics: 六个基础函数

第 11 页列出 MPI 基础函数。

11.1 MPI_Init

```
MPI_Init(&argc, &argv);
```

初始化 MPI 环境。

每个 MPI 程序都要先调用它。

11.2 MPI_Finalize

```
MPI_Finalize();
```

结束 MPI 计算。

11.3 MPI_Comm_size

```
MPI_Comm_size(comm, &size);
```

获得通信域中的进程总数。

例如：

```
MPI_Comm_size(MPI_COMM_WORLD, &numtasks);
```

11.4 MPI_Comm_rank

```
MPI_Comm_rank(comm, &pid);
```

获得当前进程编号。

例如：

```
MPI_Comm_rank(MPI_COMM_WORLD, &taskid);
```

如果有 4 个进程，rank 通常是：

```
0, 1, 2, 3
```

rank 0 通常作为主进程。

11.5 MPI_Send

```
MPI_Send(buf, count, datatype, dest, tag, comm);
```

发送消息。

参数含义：

- `buf`：发送缓冲区；
 - `count`：元素个数；
 - `datatype`：数据类型；
 - `dest`：目标进程；
 - `tag`：消息标签；
 - `comm`：通信域。
-

11.6 MPI_Recv

```
MPI_Recv(buf, count, datatype, source, tag, comm, status);
```

接收消息。

参数含义：

- `buf`：接收缓冲区；
- `count`：接收元素个数；
- `datatype`：数据类型；
- `source`：源进程；
- `tag`：消息标签；
- `comm`：通信域；
- `status`：接收状态。

这六个函数是 MPI 入门必须掌握的核心。

12. MPI 方式计算 π

第 12 页给出 MPI 版本的 π 计算。

核心思想是：

每个进程计算一部分积分区间；
每个进程得到局部和 `pi`；
最后用 `MPI_Reduce` 把所有局部和求和。

循环分配方式是：

```
for (i = taskid; i < N; i += numtasks)
```

这表示：

- rank 0 计算 `i = 0, numtasks, 2*numtasks ...`
- rank 1 计算 `i = 1, 1+numtasks, 1+2*numtasks ...`
- rank 2 计算 `i = 2, 2+numtasks, 2+2*numtasks ...`
- 以此类推。

最后：

```
MPI_Reduce(&local, &pi, 1, MPI_Double, MPI_MAX, 0, MPI_COMM_WORLD);
```

课件中写的是 `MPI_MAX`，但从计算 π 的语义看，一般应该使用 **求和归约**，也就是类似 `MPI_SUM`。
因为每个进程计算的是部分积分和，最终应该把所有部分和加起来，而不是取最大值。

rank 0 输出结果：

```
if (taskid == 0)
    printf("pi is %f\n", pi * w);
```

这体现了 MPI 程序的典型结构：

初始化 MPI
获取 `rank` 和进程数
各进程并行计算局部结果
集体通信归约结果
主进程输出
结束 MPI

13. PVM: Parallel Virtual Machine

第 13 页讲 PVM。

PVM 是 Oak Ridge National Laboratory 开发的公共领域软件系统。

它最初的目标是：

把网络上的异构 Unix 计算机组织成一个大型消息传递并行计算机。

后来也支持非 Unix 平台，如 Windows NT、Windows 95。

13.1 PVM 和 MPI 的区别

课件给出：

PVM	MPI
自包含系统	不是自包含系统
不是标准	是标准
适合异构网络环境	消息传递支持更强
需要 pvmd 守护进程	通常由 MPI 运行环境管理

PVM 系统由两部分组成：

1. `pvmd`：PVM daemon，每台机器上运行；
2. `libpvm3.a`：用户程序链接的 PVM 库。

PVM 提供：

- 进程管理函数；
- 通信函数。

14. PVM 程序计算 π

第 14 页给出 PVM 版本。

这个程序大意是：

1. 主进程启动多个任务；
2. 主进程把参数 `N` 发送给子进程；
3. 每个子进程计算自己负责的部分；
4. 每个子进程把局部结果发回；
5. 主进程收集结果并输出 π 。

程序里有典型 PVM 函数：

函数	作用
<code>pvm_mytid()</code>	获取当前任务 ID
<code>pvm_parent()</code>	获取父任务 ID
<code>pvm_spawn()</code>	创建子任务
<code>pvm_initsend()</code>	初始化发送缓冲区
<code>pvm_pkint()</code>	打包整数
<code>pvm_send()</code>	发送消息
<code>pvm_recv()</code>	接收消息
<code>pvm_upkint()</code>	解包整数
<code>pvm_reduce()</code>	归约
<code>pvm_exit()</code>	退出 PVM

PVM 的编程方式比 MPI 更像“虚拟机任务管理 + 消息通信”。

15. HPF：High Performance Fortran

第 15 页讲 HPF。

HPF 是 Fortran 90 的扩展标准，面向数据并行科学计算。

Fortran 90 本身已经支持数组语言，例如：

```
A = B + C
```

表示整个数组操作。

HPF 在此基础上加入数据分布指令，使程序员可以告诉编译器：

数组怎么分布到处理器上？
哪些循环可以并行？
哪些数据需要对齐？

HPF 支持：

- FORALL
- INDEPENDENT
- ALIGN
- DISTRIBUTE

其中：

指令	作用
FORALL	指定数组元素级并行计算
INDEPENDENT	告诉编译器循环迭代相互独立
ALIGN	指定数组之间的对齐关系
DISTRIBUTE	指定数据如何分布到处理器

HPF 最大吸引力是：

编译器负责生成通信代码。

这带来两个优点：

- 程序员可以专注于发现并行性、划分数据和映射任务；
- 探索不同并行算法更简单，很多时候只需要改变数据分布指令。

HPF 的缺点是：

- 编译器压力很大；
- 对不规则程序支持较弱；
- 性能高度依赖编译器质量。

16. HPF 中的 Gaussian Elimination

第 16 页给出 HPF 版本的高斯消去。

程序中可以看到：

```
!HPF$ PROCESSOR Nodes(4)
!HPF$ ALIGN x(i) WITH A(i,j)
!HPF$ DISTRIBUTE A(BLOCK,*) ONTO Nodes
```

这些是 HPF 指令。

含义大致是：

- 定义 4 个处理器节点；
- 把向量 $x(i)$ 和矩阵 $A(i,j)$ 对齐；
- 将矩阵 A 按 block 方式分布到处理器上。

程序中还使用：

```
pivot_location = MAXLOC(ABS(A(i:n, i)))
```

用于选主元。

然后使用：

```
FORALL(j = i+1:n, k = i+1:n+1)
    A(j,k) = A(j,k) - A(j,i) * A(i,k)
```

FORALL 表示这些更新可以并行执行。

这个例子体现了 HPF 的风格：

程序员描述数据分布和并行循环；
编译器负责具体通信和并行执行。

17. Two Ways to Parallelize Compiler

第 17 页讲编译器并行化的两条路线：

SIMDizing / Vectorizing
MIMDizing / Parallelizing

17.1 SIMDizing：向量化

SIMDizing 也叫 vectorizing。

它挖掘的是 **指令级并行性**。

典型例子是把循环转换为向量指令：

```
for (i = 0; i < n; i++)  
  A[i] = B[i] + C[i];
```

可以变成：

```
vector A = vector B + vector C
```

向量越长，潜在加速越明显。

它是向量超级计算机的重要基础。

17.2 MIMDizing：并行化

MIMDizing 也叫 parallelizing。

它挖掘的是 **任务级并行性**。

例如把代码划分成多个任务，每个处理器执行一个任务。

加速效果取决于：

实际计算时间 / 总开销

如果通信、同步、调度开销太大，加速效果就会差。

18. SIMDizing：Vectorizing

第 18 页详细讲向量化流程：

Dependence Analysis → Parallelization → Mapping

18.1 Dependence Analysis：依赖分析

目标是判断：

两个语句，或同一循环的不同迭代，能否并行执行。

依赖类型有三种。

Anti-dependence：反依赖

先读后写。

```
x = a[i];  
a[i] = y;
```

如果顺序改变，可能导致读到错误值。

Flow-dependence: 流依赖

先写后读，也叫真依赖。

```
a[i] = x;  
y = a[i];
```

后面的语句必须等前面写完。

这是最关键的依赖。

Output-dependence: 输出依赖

先写后写。

```
a[i] = x;  
a[i] = y;
```

两个写操作顺序会影响最终结果。

18.2 依赖测试

课件指出，完全依赖测试是 NP-complete，耗时且不实际。

所以实际编译器只对某些问题类别进行测试，比如：

- perfectly nested loop，完美嵌套循环；
- separable problems，可分离问题。

常见测试技术：

- GCD test
- Banerjee bound test
- λ -test

这些测试用于判断数组下标之间是否可能冲突。

18.3 Parallelization: 并行化

目标是：

把循环迭代分散到多个处理器上。

方法包括：

- 检查循环依赖；
- 把顺序循环转换为并行循环；
- 加入必要同步；
- 消除循环依赖；
- loop fusion，循环融合；
- loop distribution，循环分布。

18.4 Mapping: 映射

最后把：

```
Data and Program → Virtual processors → Physical architectures
```

也就是把数据和程序先映射到虚拟处理器，再映射到物理机器。

19. MIMDizing: Parallelizing

第 19 页讲 MIMD 并行化的三种方式。

19.1 Library Subroutines

在顺序语言基础上增加并行函数库。

例如：

- PVM Library
- MPI Library

这是最实用的方法。

19.2 New Language Constructs

扩展语言本身，加入并行结构。

例如：

- `FORALL`
- 并行循环
- 并行块

HPF 就属于这一类。

19.3 Compiler Directives

编译指令，也叫 pragmas。

在 C 中通常写成：

```
#pragma ...
```

这些指令本质上是格式化注释，用来告诉编译器如何并行化。

例如：

```
#pragma parallel
#pragma pdo
```

现代 OpenMP 也属于这种思想。

课件最后给出实践建议：

实际应用中，应使用标准 Fortran 或 C，再加标准消息传递库，如 PVM 或 MPI。

也就是说，MPI/PVM 是当时最现实、最可移植的并行编程方式。

20. Performance Analysis：性能分析

第 20 页进入性能分析。

并行程序写完后，不仅要正确，还要快。

性能分析的目的就是找出瓶颈。

基本步骤：

```
Data Collection → Data Transformation → Data Visualization
```

也就是：

1. 收集性能数据；
2. 转换和整理数据；
3. 可视化显示数据。

并行性能分析困难的原因：

- 数据复杂；
- 数据量巨大；
- 数据多维；
- 包含执行时间、通信成本、同步等待、负载不均衡等；
- 工具标准少；
- 不同工具格式和显示方式不同。

性能工具需要考虑：

问题	含义
Accuracy	数据是否准确，取决于采样、时钟等
Simplicity	是否能自动收集，程序员干预少
Flexibility	是否能扩展，是否提供多种视图
Intrusiveness	采集数据是否会干扰程序运行
Abstraction	是否能在合适抽象层次展示数据

其中 **intrusiveness** 很重要。

性能测量本身会引入开销，可能改变程序行为，这叫 probe effect。

21. Data Collection：数据收集

第 21 页讲性能数据收集。

常见技术有三种：

```
Profiles
Counters
Traces
```

21.1 Profiles：概要分析

Profiles 记录程序在不同部分花费的时间。

特点：

- 成本较低；
- 通常自动收集；
- 应该优先考虑；
- 适合找热点函数或热点代码段。

例如：

```
函数 A: 60%
函数 B: 25%
函数 C: 15%
```

可以快速看出程序主要耗时在哪里。

21.2 Counters：计数器

Counters 记录事件发生次数或累计时间。

例如：

- 函数调用次数；
- 消息发送次数；
- cache miss 次数；
- 总通信时间；
- barrier 次数。

计数器本质上是一个存储位置，每次事件发生就增加。

缺点是有时需要程序员插入计数代码。

21.3 Traces：事件轨迹

Traces 记录指定事件的每一次发生，通常带时间戳。

例如：

```
time 0.001: process 0 send message
time 0.002: process 1 receive message
time 0.003: process 2 enter barrier
```

Trace 是最详细、最低层的性能数据收集方式。

优点：

- 能看到程序行为细节；
- 适合研究通信、同步、等待等问题。

缺点：

- 数据量巨大；
- 容易产生 probe effect；
- 分析成本高。

课件结论：

```
profile 和 counter 更容易获得和分析；
trace 能显示细节，但要谨慎使用。
```

22. Data Transformation and Visualization

第 22 页讲数据转换和可视化。

原始性能数据通常不能直接回答问题。

例如 trace 数据可能是几百万条事件记录，我们真正想知道的是：

哪个处理器空闲最多？
哪段代码通信最多？
哪个 `barrier` 等待最长？
通信模式是什么？

所以需要数据转换。

转换目标包括：

- 降低数据量；
- 计算平均值；
- 计算高阶统计量；
- 从 trace 中提取 profile 或 counter 信息。

显示格式包括：

数据类型	显示方式
0 维标量	单个数值
1 维数据	histogram，直方图
2 维数据	color matrix，二维颜色矩阵
trace 数据	histogram、Gantt Chart、space-time diagram

其中 Gantt 图和 space-time diagram 对并行程序非常有用，因为可以显示每个处理器在不同时间做什么。

23. 性能分析工具示例

第 23 页列出几个工具。

23.1 Paragraph

Paragraph 是 Oak Ridge National Laboratory 开发的可移植、交互式 trace 分析和可视化工具。

主要用于消息传递程序。

它可以显示：

- processor utilization，处理器利用率；
- communication volumes，通信量；
- communication patterns，通信模式。

缺点是性能数据和源代码之间的关系不总是清楚。

23.2 Upshot

Upshot 是 Argonne National Laboratory 开发的 trace 分析和可视化工具。

也是面向消息传递程序。

特点：

- 显示事件状态；
- 显示数量比 Paragraph 少；
- 支持滚动和缩放，这一点很有用。

23.3 ParAide

ParAide 是 Intel Supercomputer System Division 开发的系统，专门用于 Paragon 并行计算机。

它包括：

- per-node profiling，每个节点的 profiling；
- 处理器利用率可视化；
- 通信网络利用率；
- 内存总线利用率。

23.4 IBM Parallel Environment

IBM AIX Parallel Environment 面向 IBM 计算机，尤其是 SP multicomputer。

它可以使用 prof/gprof 的变体生成多个 profile 数据，也可以用 VT 显示 trace 数据。

这一部分说明：

并行性能工具通常和具体机器、编程模型密切相关。

24. Scientific Data Visualization：科学数据可视化

第 24 页开始讲科学数据可视化。

定义：

科学数据可视化是用图形方法增强对科学数据的解释和理解。

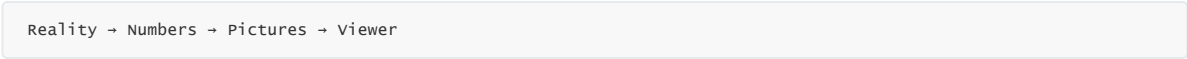
目标：

提供概念、方法和工具，从科学数据中创建表达力强、有效的视觉表示。

科学可视化涉及多个学科：

- 心理学 / 感知；
- 计算机图形学；
- 艺术；
- 图形设计。

第 24 页图中给出了一个映射过程：



也就是：

1. 真实现象；
2. 通过测量或模拟变成数字；
3. 数字通过图形方法变成图片；
4. 观众解释图片。

这说明科学可视化不只是画图，而是从真实世界到主观理解的完整映射过程。

25. Scientific Data 的特征

第 25 页讲科学数据的视觉线索和数量类型。

25.1 Visual Cues：视觉线索

视觉线索是图像中的表达元素。

包括：

类型	例子
Spatial	位置、运动
Shape	长度、深度、面积、体积、厚度
Orientation	角度、斜率
Density	密度
Color	颜色、对比度

不同数据类型适合不同视觉线索。

例如：

- 数值大小可以用长度、颜色、面积表达；
- 方向可以用箭头表达；
- 密度可以用灰度或颜色深浅表达。

25.2 Quantitative Type：数量类型

课件列出几类科学数据。

Point data: 点数据

形式类似:

```
(x_i, y_i)
```

适合用:

- scatter plot, 散点图;
- glyph, 符号图。

Scalars: 标量

标量通常是连续函数的采样:

```
y_i = f_i(x_1, x_2, ..., x_n)
```

例如温度、压力、浓度。

适合用:

- 曲线图;
- 等值线;
- 颜色图;
- 表面图。

Vectors: 向量

向量有大小和方向。

例如速度场、力场。

适合用箭头显示:

```
箭头方向 = 向量方向
箭头长度 = 向量大小
```

Tensor field: 张量场

张量是向量和矩阵的扩展。

在 n 维张量场中, 需要计算:

- principal directions, 主方向;
- magnitudes, 大小。

张量场可视化更复杂, 常见于流体、材料、医学图像等领域。

26. Visualization Techniques: 可视化技术

第 26 页介绍多种可视化方法。

26.1 Scatter Plots: 散点图

散点图主要使用 **position, 位置** 作为视觉线索。

通常用于点数据。

例如:

```
横轴 x, 纵轴 y, 每个点表示一个样本。
```

有时会使用对数尺度, 以显示跨度很大的数据。

26.2 Glyphs: 符号图

Glyph 是由多个视觉部件组成的小图形, 用于表达复杂数据。

例如一个 glyph 可以包含:

- 两个圆;
- 一条竖线;
- 亮度;
- 曲线;
- 方向;

- 大小。

观众通常把这些部件识别为一个整体图形。

Glyph 适合多变量数据，例如每个点上同时有温度、速度、压力、方向等多个属性。

26.3 Linear Graphs：线图

线图用于显示连续信息，是标量数据集的有效表示方式。

例如：

时间 - 温度曲线
迭代次数 - 误差曲线
处理器数量 - 加速比曲线

26.4 Histograms：直方图

直方图类似离散线图或柱状图。

每个矩形的面积有特殊意义，通常表示频数或概率。

例如：

运行时间分布
消息长度分布
误差分布

26.5 Pie Charts：饼图

当数据需要和总量比较时，可以用饼图。

例如：

计算时间占比
通信时间占比
I/O 时间占比
同步等待占比

但饼图适合类别较少的情况，类别太多会不清楚。

26.6 Contour Plot：等值线图

Contour plot 也叫 isolines，等值线。

它用于二维数据集中显示常数值线。

例如地形图中的等高线：

所有高度相同的点连成一条线。

科学计算中可以用来显示：

- 温度等值线；
- 压力等值线；
- 电势等值线。

27. More About Visualization Techniques

第 27 页继续介绍可视化技术。

27.1 Image Display：图像显示

常用于二维定量数据。

用灰度或颜色表示数据值。

例如：

黑色 = 小值
白色 = 大值
或用颜色渐变表示数值大小

适合：

- 医学图像；
- 热力图；
- 矩阵数据；
- 二维模拟结果。

27.2 Surface View：表面视图

把数据值看作高度，显示成地形表面。

例如：

局部最大值 = 山峰
局部最小值 = 山谷

适合显示二维标量函数：

$z = f(x, y)$

27.3 Iso Surfaces：等值面

如果数据在三维规则网格上，可以用等值面表示体数据。

它是等值线的三维版本。

等值线：二维中相同值的线
等值面：三维中相同值的面

适合：

- 医学 CT/MRI；
- 流体模拟；
- 三维温度场；
- 三维密度场。

27.4 Ray Tracing of Volumes：体数据光线追踪

如果想显示体内部所有元素，可以用 ray tracing。

体数据中的每个小单元叫 voxel，体素。

光线追踪把三维体素投影到二维图像上，可以产生透明、云状的效果。

适合显示：

- 云团；
- 烟雾；
- 医学体数据；
- 体密度分布。

27.5 Animation：动画

动画通过连续图片展示变化过程。

适合显示：

- 时间变化；
- 动态模拟；
- 流体运动；
- 迭代过程；
- 物理场演化。

动画的优势是能让观察者看到数据如何随时间变化，而不是只看静态结果。

28. 本章总复习

Part III 的核心可以这样总结：

并行编程模型

- └─ **Implicit**: 写顺序程序，编译器自动并行化
- └─ **Data-Parallel**: 对大量数据执行相同操作
- └─ **Shared-Variable**: 多线程通过共享变量通信
- └─ **Message-Passing**: 多进程通过消息通信

编程工具和标准

- └─ **ANSI X3H5**: 共享内存并行结构
- └─ **Pthreads**: 线程创建、互斥锁、条件变量
- └─ **MPI**: 标准消息传递接口
- └─ **PVM**: 异构网络上的并行虚拟机
- └─ **HPF**: 数据并行 **Fortran** 扩展

编译器支持

- └─ **SIMDizing / Vectorizing**: 指令级并行，向量化循环
- └─ **MIMDizing / Parallelizing**: 任务级并行，库、语言结构、**pragma**

性能分析

- └─ **Data Collection**: **profile**、**counter**、**trace**
- └─ **Data Transformation**: 统计、压缩、提取信息
- └─ **Data Visualization**: 直方图、**Gantt** 图、时空图

科学可视化

- └─ 点数据: **scatter plot**、**glyph**
- └─ 标量场: 线图、等值线、图像显示、表面图
- └─ 向量场: 箭头图
- └─ 张量场: 主方向和大小
- └─ 体数据: 等值面、体绘制、动画

29. 最重要复习点

这一章最应该掌握：

1. 四种并行编程模型的区别

Implicit: 自动并行

Data-Parallel: 数据并行

Shared-Variable: 共享内存

Message-Passing: 显式消息通信

1. 共享变量模型和消息传递模型的核心差异

共享变量：单地址空间，隐式通信，显式同步

消息传递：多地址空间，显式通信，显式数据分配

1. Pthreads 的核心机制

线程创建 + join + mutex + condition variable

1. MPI 六个基础函数

MPI_Init

MPI_Finalize

MPI_Comm_size

MPI_Comm_rank

MPI_Send

MPI_Recv

1. MPI 计算 π 的并行思想

循环分块/交错分配 → 每个进程局部求和 → **Reduce** 汇总

1. MPI 和 PVM 的区别

MPI 是标准，消息传递支持强；

PVM 是自包含系统，适合异构虚拟机环境。

1. HPF 的核心思想

用数据分布指令描述并行，编译器生成通信代码。

1. 向量化中的依赖分析

反依赖、流依赖、输出依赖

1. 性能分析三类数据收集方法

Profile: 低成本热点分析

Counter: 事件计数

Trace: 最详细但数据量大

1. 科学可视化的映射链条

Reality → Numbers → Pictures → Viewer

这一章整体是在讲：

并行程序不仅要有硬件和算法，还需要合适的编程模型、编程标准、编译器支持、性能分析工具和可视化方法。

下面继续讲你上传的 **PC0.pdf**。这份课件不是新的技术章节，而是整门 **Parallel Computing** 课程的总览/导论，相当于把前面 Part I、Part II、Part III 三大部分串起来。内容来自 `PC0.pdf`。

PC0: Parallel Computing 课程总览

1. 课程主题：Parallel Computing

第 1 页是封面：

Parallel Computing
Guoliang Chen
University of Science and Technology of China

也就是陈国良老师的并行计算课程。

这门课的核心不是只讲“怎么写并行程序”，而是从三个层面系统讲并行计算：

Parallel Computing
├─ Parallel Architectures
├─ Parallel Algorithms
└─ Parallel Programming

也就是第 3 页列出的三大主题：

1. 并行体系结构
2. 并行算法
3. 并行编程

可以理解为：

硬件平台 → 算法理论 → 编程实现

2. 并行计算课程体系图

第 2 页是中文课程体系图，内容比较密集。

它把并行计算相关课程放在一个大的知识体系中，包括：

算法理论
├─ 可计算性与计算复杂性
└─ 算法研究的数学基础

算法设计与分析
├─ 并行排序和选择算法
└─ VLSI 计算理论与并行算法

|— 并行图论算法
|— 分布式算法
└— 并行算法的设计与分析

算法实现
|— 并行算法实践
└— 并行计算机体系结构

算法应用基础
|— 并行计算：结构、算法、编程
└— 高性能计算导论

这张图的意思是：

并行计算不是孤立课程，它连接了算法理论、算法设计、体系结构、程序实现和高性能计算应用。

也就是说，要真正理解并行计算，需要同时理解：

- 计算机体系结构；
- 算法复杂度；
- 并行算法设计；
- 分布式系统；
- 并行程序设计；
- 高性能计算应用。

3. 课程摘要 Abstract

第 4 页是整门课的摘要。

课件说，并行计算一般包括三个方面：

parallel computer architectures
parallel algorithms
parallel programming

对应课程三部分：

部分	内容
Part I	并行计算机系统结构、存储访问模型、系统互连、性能评价
Part II	并行计算模型、并行算法设计方法/技巧/方法论、并行数值算法
Part III	并行编程模型、共享内存编程、消息传递编程、数据并行编程、环境和工具

这也正好对应你前面上传的三份课件：

Part I → 硬件与体系结构
Part II → 算法与计算模型
Part III → 编程模型与工具

4. Part I: Parallel Computer Systems

第 5 页说明 Part I 是并行计算的 **Hardware Platform**，即硬件平台。

主要包括：

Part I: Parallel Computer Systems
|— System Architectures and Models
|— System Interconnections
└— Performance Evaluation

也就是：

1. 并行计算机系统结构和模型；
2. 并行系统互连网络；
3. 并行系统性能评价。

这一部分回答的问题是：

并行程序到底运行在什么机器上？
这些机器如何组织处理器、内存和网络？
机器性能如何评价？

4.1 System Architectures and Models

第 6 页列出 Part I 的体系结构与存储模型。

并行计算机系统结构

包括：

PVP: Parallel Vector Processors
SMP: Symmetric Multiprocessors
MPP: Massively Parallel Processors
DSM: Distributed Shared Memory
COW: Cluster Of Workstations

简单回顾：

类型	核心特点
PVP	少量强大的向量处理器，共享内存
SMP	多个对称处理器，共享内存
MPP	大量节点，分布式内存，专用网络
DSM	物理分布内存，逻辑共享地址空间
COW	普通工作站/PC 通过网络组成集群

这一组内容对应 Part I 开头讲的并行计算机类型。

并行计算机存储访问模型

包括：

UMA: Uniform Memory Access
NUMA: Non-Uniform Memory Access
COMA: Cache-Only Memory Access
NORMA: NO-Remote Memory Access

简单回顾：

模型	含义
UMA	所有处理器访问内存时间相同
NUMA	访问本地内存快，远程内存慢
COMA	内存全部 cache 化，数据可迁移
NORMA	不能远程访存，只能消息通信

这一部分的重点是：

体系结构决定编程模型和性能瓶颈。

例如：

- SMP/UMA 更适合共享内存编程；
- MPP/NORMA 更适合 MPI 消息传递；
- DSM/NUMA 想兼顾共享内存便利和分布式扩展性。

4.2 System Interconnections

第 7 页讲系统互连。

并行计算机之所以复杂，很大原因在于多个处理器之间必须交换数据。
所以互连网络非常重要。

课件分成三层：

Network Environments

- └─ Intra-node Interconnections: 节点内部互连
- └─ Inter-node Interconnections: 节点之间互连
- └─ Inter-system Interconnections: 系统之间互连

对应：

层次	例子
节点内部	buses, switches
节点之间	SAN
系统之间	LAN, MAN, WAN

然后讲互连拓扑：

Interconnection Topologies

- └─ Static-Connection Networks
- └─ Dynamic-Connection Networks

静态连接网络包括：

LA, RC, MC, TC, HC, CCC

可以理解为：

缩写	常见含义
LA	Linear Array, 线性阵列
RC	Ring Connection, 环
MC	Mesh Connection, 网格
TC	Tree Connection, 树
HC	Hypercube, 超立方体
CCC	Cube-Connected Cycles

动态连接网络包括：

Buses
Crossbar
MIN

其中 MIN 是 Multistage Interconnection Network, 多级互连网络。

宽带网络包括：

FDDI
FE/GE
ATM
SCI

也就是：

技术	全称
FDDI	Fiber Distributed Data Interface
FE/GE	Fast Ethernet / Gigabit Ethernet
ATM	Asynchronous Transfer Mode
SCI	Scalable Coherence Interface

这一部分的核心是：

并行系统性能不仅取决于 CPU，还强烈取决于互连网络的延迟、带宽、拓扑和拥塞。

4.3 Performance Evaluation

第 8 页讲性能评价。

包括三类内容：

```
Performance Evaluation
├── Speed up of Systems
├── Scalability of Systems
└── Performance of Systems: Benchmarks
```

加速比定律

包括：

```
Amdahl's Law
Gustafson's Law
Sun and Ni's Law
```

简单回顾：

定律	关注点
Amdahl	固定问题规模，串行部分限制加速比
Gustafson	固定时间，处理器越多解决越大问题
Sun-Ni	内存容量随处理器增长，解决受内存限制的大问题

可扩展性指标

包括：

```
Iso-efficiency
Iso-speed
Average Latency
```

简单理解：

指标	说明
Iso-efficiency	为保持效率，问题规模要如何增长
Iso-speed	为保持平均速度，问题规模和机器规模如何变化
Average Latency	从平均延迟角度评价系统扩展

Benchmark

课件列出：

```
LINPACK
SPEC
PARKBENCH
NAS
```

Benchmark 的作用是用标准测试程序评价系统性能。

例如：

- LINPACK 常用于线性代数性能；
- SPEC 常用于处理器/系统性能；
- NAS 常用于并行计算基准测试。

5. Part II: Parallel Algorithms

第 9 页说明 Part II 是并行计算的 **Theoretical Base**，即理论基础。

内容包括：

Part II: Parallel Algorithms

- └─ Computational Models
- └─ Design Policy
- └─ Design Techniques
- └─ Design Methodology
- └─ Parallel Numerical Algorithms

这一部分回答的问题是：

怎样抽象并行计算？
怎样设计并行算法？
怎样分析复杂度？
怎样把经典数值算法并行化？

5.1 Computational Models

第 10 页列出并行计算模型：

PRAM
APRAM
BSP
LogP

简单回顾：

模型	含义	特点
PRAM	Parallel Random Access Machine	理想共享内存，同步，适合理论分析
APRAM	Asynchronous PRAM	异步 PRAM，引入 phase 和 barrier
BSP	Bulk Synchronous Parallel	superstep，本地计算 + 全局通信 + barrier
LogP	Latency, Overhead, Gap, Processors	更细致描述消息通信代价

模型从 PRAM 到 LogP，越来越接近真实机器：

PRAM：最理想，最适合理论
APRAM：加入异步和显式同步
BSP：适合结构化并行算法设计
LogP：更关注底层通信开销

5.2 Design Policy

第 11 页讲并行算法设计策略。

包括三种：

Parallelizing a Sequential Algorithm
Designing a new Parallel Algorithm
Borrowing other well-known Algorithm

也就是：

方法	含义
并行化顺序算法	从已有顺序算法中挖掘并行性
设计新的并行算法	从问题本身出发重新设计
借用经典算法	把问题转化为已知问题，借用成熟算法思想

这一页的重点是：

好的顺序算法不一定是好的并行算法。

并行算法设计要考虑：

- 任务能不能同时做；
- 通信代价大不大；
- 负载是否均衡；
- 同步是否频繁；
- 可扩展性如何。

5.3 Design Techniques

第 12 页列出并行算法设计技巧：

Balanced Trees
Doubling Technique
Partitioning Strategy
Divide and Conquer
Pipelining

简单回顾：

技巧	典型用途
Balanced Trees	前缀和、归约、最大最小值
Doubling Technique	pointer jumping、链表排名、找树根
Partitioning Strategy	数据划分、任务划分
Divide and Conquer	递归分解并行
Pipelining	流水线、阶段化计算

这些技巧是并行算法设计的常用“套路”。

例如：

- 求和可以用 Balanced Tree；
- 链表问题可以用 Doubling；
- 矩阵/网格问题可以用 Partitioning；
- FFT 类问题常用 Divide-and-Conquer；
- 多阶段处理可以用 Pipelining。

5.4 Design Methodology

第 13 页讲 PCAM 方法论：

PCAM
├─ Partitioning
├─ Communication
├─ Agglomeration
└─ Mapping

这四步非常重要：

阶段	作用
Partitioning	把问题分成很多可并行任务
Communication	分析任务之间需要交换哪些数据
Agglomeration	合并任务，调整粒度，减少通信
Mapping	把任务分配到处理器上

可以记成：

先拆开 → 看通信 → 再合并 → 放到机器上

5.5 Parallel Numerical Algorithms

第 14 页列出并行数值算法主题：

```
Dense Matrix Algorithms
Solving Systems of Linear Equations
Fast Fourier Transform
```

也就是：

- 1. 稠密矩阵算法；
- 2. 线性方程组求解；
- 3. 快速傅里叶变换。

这些是高性能计算中最重要的数值算法基础。

前面 Part II 详细讲过：

- Cannon 矩阵乘法；
- DNS 矩阵乘法；
- Gauss-Seidel 并行化；
- 红黑着色；
- 分治/流水线等技巧。

6. Part III: Parallel Programming

第 15 页说明 Part III 是并行计算的软件支持部分。

```
Part III: Parallel Programming
├─ Programming Models
├─ Shared-Memory Programming
├─ Message-Passing Programming
├─ Data-Parallel Programming
└─ Programming Environment and Tools
```

这一部分回答的问题是：

```
并行算法最终怎么写成程序？
用什么编程模型？
用什么库？
怎么调试？
怎么分析性能？
```

6.1 Programming Models

第 16 页列出四种编程模型：

```
Implicit Model
Data-Parallel Model
Shared-Memory Model
Message-Passing Model
```

简单回顾：

模型	核心特点
Implicit	程序员写顺序程序，编译器自动并行化
Data-Parallel	对大量数据执行相同操作
Shared-Memory	多线程共享地址空间，通过共享变量通信
Message-Passing	多进程独立地址空间，通过消息通信

它们和体系结构有对应关系：

SMP/DSM → Shared-Memory
MPP/COW → Message-Passing
SIMD/数组程序 → Data-Parallel
自动并行编译 → Implicit

6.2 Shared-Memory Programming

第 17 页讲共享内存编程。

包括：

ANSI X3H5
POSIX Threads(Pthreads)
OpenMP
Shared-Variable Parallel Code to Compute Pi

这里 OpenMP 在 Part III 详细课件中没有展开，但它和 ANSI X3H5 的思想类似：
通过编译指令指定并行区域、并行循环、共享/私有变量和同步。

共享内存编程的核心是：

多个线程 + 一个共享地址空间 + 显式同步

典型问题是：

- 数据竞争；
- 临界区；
- 锁；
- barrier；
- false sharing；
- 负载不均衡。

计算 π 的共享变量版本体现了：

并行循环 + 局部计算 + 临界区归约

6.3 Message-Passing Programming

第 18 页讲消息传递编程。

包括：

MPI: Message-Passing Interface
MPI Basics
Message-Passing Code to Compute Pi
PVM: Parallel Virtual Machine
PVM Program to Compute Pi

MPI 是标准消息传递接口，PVM 是并行虚拟机系统。

消息传递模型的核心是：

多个进程 + 多地址空间 + 显式通信

MPI 程序典型结构是：

MPI_Init
获取 rank 和 size
每个进程计算局部数据
Send/Recv 或 Reduce/Bcast
rank 0 汇总输出
MPI_Finalize

计算 π 的 MPI 版本体现了：

6.4 Data-Parallel Programming

第 19 页讲数据并行编程。

包括：

HPF: High-Performance Fortran
Gaussian Elimination in HPF

HPF 的思想是：

程序员用数组语言和数据分布指令描述并行，编译器负责生成通信代码。

常见指令包括：

FORALL
INDEPENDENT
ALIGN
DISTRIBUTE

数据并行适合：

- 数组；
- 矩阵；
- 网格；
- 科学计算；
- 规则数据结构。

6.5 Programming Environment and Tools

第 20、21 页讲编程环境和工具。

第 20 页主要讲：

Parallelizing Compiler
├─ Two Ways to Parallelize Compiler
├─ SIMDizing / Vectoring
├─ MIMDizing / Parallelizing
└─ Dependency Analysis

Performance Analysis
├─ Performance Prediction
├─ Performance Monitoring
└─ Performance Visualization

并行化编译器

有两类：

类型	含义
SIMDizing / Vectoring	向量化，挖掘指令级并行
MIMDizing / Parallelizing	并行化，挖掘任务级并行

关键是 Dependency Analysis，依赖分析。
编译器必须判断循环迭代之间有没有数据依赖，能不能并行执行。

性能分析

包括：

Performance Prediction: 性能预测
Performance Monitoring: 性能监控
Performance Visualization: 性能可视化

并行程序性能分析比顺序程序更难，因为要看：

- 计算时间；
- 通信时间；
- 同步等待；
- 负载均衡；
- 处理器利用率；
- 消息模式；
- 内存访问。

第 21 页继续讲工具：

Parallel Program Debugging
└─ Correct Debugging
└─ Performance Debugging

Graphical Development Environment for Parallel Programming
└─ Graphical Editor and Language
└─ Pre-Compilation
└─ Mapping
└─ Debugging and Monitoring

这里区分了两类调试：

类型	目标
Correct Debugging	程序结果是否正确
Performance Debugging	程序为什么不够快

并行程序调试难点是：

- 执行顺序不确定；
- 多线程/多进程交互复杂；
- 死锁难复现；
- race condition 难定位；
- 性能瓶颈可能来自通信或同步，而不是计算。

图形化开发环境的目标是帮助程序员：

编辑程序 → 预编译 → 映射任务 → 调试 → 监控运行

7. 第 22 页：结束页

第 22 页是结束页：

Thank you for your listening!

这说明 PC0 是课程引导课件，重点是帮助学生建立并行计算课程的整体框架。

8. 这份 PC0 与前三份课件的关系

可以把四份课件关系理解成：

PC0.pdf: 总目录 / 课程导论
└─ partI.pdf: 并行计算机系统
└─ partII.pdf: 并行算法
└─ partIII.pdf: 并行编程

PC0 只是把课程地图列出来，而 Part I、II、III 是具体展开。

9. 最重要的总框架

整门课可以用这张结构记忆：

```
Parallel Computing
├── 1. Hardware: Parallel Architectures
│   ├── PVP, SMP, MPP, DSM, COW
│   ├── UMA, NUMA, COMA, NORMA
│   ├── Bus, Crossbar, MIN, Mesh, Hypercube
│   └── Amdahl, Gustafson, Sun-Ni, Scalability
│
├── 2. Theory: Parallel Algorithms
│   ├── PRAM, APRAM, BSP, LogP
│   ├── 并行化顺序算法 / 重新设计 / 借用经典算法
│   ├── Balanced Tree, Doubling, Partitioning, D&C, Pipelining
│   ├── PCAM
│   └── Cannon, DNS, Gauss-Seidel, FFT
│
└── 3. Software: Parallel Programming
    ├── Implicit, Data-Parallel, Shared-Memory, Message-Passing
    ├── Pthreads, OpenMP, ANSI X3H5
    ├── MPI, PVM
    ├── HPF
    └── 编译器、调试、性能分析、可视化
```

10. 复习时怎么抓重点

这份 PC0 最适合作为总复习提纲。你可以按下面三句话记：

Part I 讲“机器”：处理器、内存、互连网络、性能评价。

Part II 讲“算法”：计算模型、设计策略、设计技巧、PCAM、数值算法。

Part III 讲“程序”：编程模型、MPI/PVM/Pthreads/HPF、编译器、调试和性能分析。

整门课的核心逻辑就是：

先理解并行机器怎么组织，
再理解并行算法怎么设计，
最后理解并行程序怎么实现和优化。